Test-Driven Exploration

Testing a legacy codebase starts in earnest when there is a change to make. Often the first order of business on a new project is to deal with a critical bug left by the previous team—something that must be accomplished while preserving existing behavior and that does not demand a dramatic refactoring of the application.

In this case, there are two goals to getting the code under test. You want to be able to tell when the bug has been fixed. This step involves a more or less standard TDD bug-fixing session with one or more failing tests isolating the bug. The tests should pass when the bug is fixed. Also, you must confirm that any existing correct behavior hasn't been compromised. In a project that was TDD from the beginning you'd already have this ability, but in a testless legacy project you need to build up that coverage.

Generally speaking, tests against an existing system come in two flavors: black-box and white-box testing—the phrases far predate software testing and apply to any kind of test process. A black-box test is so called because it ignores the internal structure of the application and tests only top-level input into the system and the output that is returned. Conversely, a white-box test uses knowledge about system internals to explicitly test specific paths through the code.

Black-Box Testing

A *black-box* test is a test that depends only on the external interface of code without relying on the internals. You can think of the internals as an opaque black box that you can't see inside. We already did black-box testing of Rails applications back in Chapter 8, *Integration Testing with Capybara and Cucumber*, on page 151. A black-box test of a Rails application works only at the level of user input and system output, typically HTML. That sounds a lot like a Cucumber or Capybara integration test. Since integration testing works from outside the normal Rails code, it's ideal for interaction with legacy code.

The integration tests interact with the system as a user would. Since there is no interaction with the code's internal structure, it's possible to write integration tests no matter how gunky the code is.

Integration testing can be useful in a bug situation because bugs are often specified in terms of the users' actions and responses. These actions and responses are often reasonably straightforward to translate to Cucumber or Capybara, and it's easy to recognize if you've changed the behavior. In addition,